Developer Guide for AWS CI CD

A proposed development, staging and production release process using GitHub Actions

Audience: developers, tech leads and DevOps engineers. This guide explains how a backend API on EC2 and a static frontend on S3 move safely from development to production, with tech-lead review before staging and production promotion.

Implementation status: the current challenge workflows provide PR checks and production deployment. The three-environment process and separate promotion repositories described here are a proposed extension; they are not yet implemented or validated in AWS.

1 Current problem and proposed solution

Today	Proposed process
Backend and frontend are released manually; steps may vary by developer.	GitHub Actions builds and tests releases, then deploys through consistent procedures.
A manual release may miss a test, use the wrong version or be hard to trace.	An immutable release identifies the source commit, tested backend image and frontend build.
Staging and production changes lack a consistent review path.	A tech lead reviews promotion pull requests into protected staging and production repositories.
Recovery depends on remembering previous steps.	Document the previous known-good release, rollback and service verification.

Expected outcome: faster, repeatable and more secure delivery with fewer avoidable errors and clear human accountability. No pipeline can guarantee error-free code or zero incidents; validation, monitoring and rollback remain necessary.

2 Environments and applications

Environment	Purpose	AWS resources and data
Development	Develop and validate changes.	Dev AWS account preferred; API on dev EC2, SPA on dev S3 / CloudFront, separate secrets and synthetic data.
Staging	Test the same release in a production-like environment.	Separate staging account preferred; staging EC2, S3 / CloudFront, IAM role and non-production data.
Production	Serve real users.	Separate production account; production EC2, S3 / CloudFront, restricted IAM, monitoring and backups.

Each environment uses its own account or equivalent isolation, IAM roles, secrets, resource names and endpoints. No production credentials or customer data in dev/staging without a separately approved process. Staging should mirror production architecture and controls as closely as practical.

3 Repository and review model

For the requested model, we propose three GitHub repositories. The staging and production repositories hold release configuration and immutable artifact references, not independent copies of application source code.

Repository	Contents	Who reviews
application-repo	Backend and frontend source, tests, Dockerfile and build workflow.	Normal team review before merging code to main.

Repository	Contents	Who reviews
staging-deploy-repo	Staging deployment workflow, environment configuration and release manifest.	Tech lead reviews the promotion PR into its protected main branch.
production-deploy-repo	Production deployment workflow, environment configuration and release manifest.	Tech lead reviews the promotion PR into its protected main branch.

This guide assumes these repositories and their automation will be created. Configure rulesets on staging and production main: require passing checks, a tech-lead code-owner review, dismissal of stale approvals where appropriate, and restrictions on bypass/direct pushes. CODEOWNERS must exist on the PR base branch; permissions and GitHub plan features must be checked. If a tech lead is unavailable, use an explicitly authorized delegate under policy.

4 The new CI CD workflow

Step	Owner / GitHub Action	Result
1 Develop	Developer creates a branch and commits changes.	Work isolated from main.
2 Code PR	Open PR to application-repo main; Actions runs checks; team reviews.	Only reviewed, passing code can merge.
3 Build candidate	After merge, Actions builds and tests the exact merge commit.	One backend image digest and one frontend artifact checksum.
4 Deploy DEV	Actions deploys the candidate to development; runs smoke and integration checks.	Candidate is validated in dev.
5 Staging PR	Developer or automation opens a PR in staging-deploy-repo containing the exact candidate references and dev evidence.	Tech lead reviews the promotion; required checks must pass.
6 Deploy STAGING	After tech-lead approval and PR merge, staging Actions deploys those exact artifacts.	Staging tests and evidence are linked to the release.
7 Production PR	A PR in production-deploy-repo contains the same references and successful staging evidence.	Tech lead reviews and approves before merge.
8 Deploy PROD	After PR merge and any required environment approval, production Actions deploys and validates.	Users receive the reviewed release.
9 Monitor or roll back	DevOps and developers check customer path, errors and latency.	Healthy release recorded, or known-good version restored.

Sequence: code PR → build once → DEV tests → staging promotion PR and tech-lead approval → STAGING tests → production promotion PR and tech-lead approval → PROD verification.

A PR approval for a repository merge and a GitHub Environment deployment approval are different controls. Require both only if desired and available; identify the responsible reviewers and plan the process for private repositories. The tech lead cannot approve a failed test into success: the gate must pass or an explicitly governed exception must be recorded.

5 Step by step for developers

Step 1 Start a feature

```
git switch main
git pull origin main
git switch -c feature/short-description
```

Implement the change in application-repo. Update tests and configuration documentation. Never commit secrets, access keys or environment files containing credentials.

Step 2 Test locally and open the code PR

```
cd backend
npm ci
npm run lint
npm test
cd ../frontend
npm ci
npm run lint
npm test
npm run build
```

These commands are proposed contracts: they work only when the real repositories provide the corresponding package files and scripts. Open a PR to application-repo main; Actions repeats the checks. Add the purpose, impact and test evidence. A reviewer approves and merges only after checks pass.

Step 3 Build and validate in DEV

After the merge, CI builds the backend container and frontend once. It records the application commit SHA, container image digest, frontend artifact checksum and build run. The deploy workflow assumes the dev AWS role, deploys the backend digest through SSM to dev EC2, publishes the frontend artifact to the dev S3 bucket, invalidates the dev CloudFront distribution and runs smoke/integration checks.

If DEV fails, stop promotion. Investigate the failing logs, correct code or configuration, and produce a new candidate. Never use a production AWS role for a dev job.

Step 4 Request staging promotion

Open a PR against staging-deploy-repo main that updates the release manifest with the already tested image digest and frontend artifact checksum. Include the source commit, dev test run, compatibility notes and rollback target. The PR does not deploy production or rebuild the application.

The tech lead checks the exact release, tests, security findings, dependencies and rollback plan. After required checks and approval, merge the PR. Staging Actions assumes only the staging role and deploys the exact candidate.

Step 5 Run staging tests

- Verify frontend and API health through the actual staging entry points.
- Run API contract, integration and end-to-end customer-flow tests using synthetic data.
- Run security, dependency and representative performance checks; verify the rollback procedure.
- Record the test results and release identifiers. If any mandatory check fails, stop and fix the issue before a production PR.

Step 6 Request production promotion

Open a PR against production-deploy-repo main using the same approved release references. Attach staging evidence and the proposed release and rollback plan. The tech lead validates and approves the PR; protected-branch rules prevent merge until required reviews and checks pass.

After merge, production Actions applies any configured GitHub Environment deployment gate, assumes the production AWS role and deploys the reviewed artifacts. The release owner verifies the public API and frontend, then monitors the agreed observation window.

Step 7 Handle failure

Stop further promotion, alert the owner and restore the previous known-good artifact using the applicable runbook. Verify rollback through the public path. A single EC2 deployment may briefly interrupt service; do not describe it as zero downtime. Frontend index rollback does not undo database changes. Record the incident and permanent follow-up.

6 GitHub Actions pipeline behavior

Pipeline stage	Checks / deployment	If it fails
Pull request CI	Install locked dependencies; lint, unit and integration tests; security and dependency scans.	Do not merge. PR jobs have no AWS production credentials.
Build	Build and smoke-test the backend image; build and validate the frontend artifact; record immutable IDs.	Do not create a promotable release.
DEV deploy and test	OIDC dev role; EC2/SSM and S3/CloudFront deploy; integration tests.	Stop and repair DEV.
STAGING promotion	Tech-lead PR approval; deploy exact artifacts; run acceptance tests.	Stop and create a corrected candidate.
PROD promotion	Tech-lead PR approval; configured deployment approval; deploy and verify.	Stop release, execute and verify rollback, raise incident if needed.

7 Artifact identity and configuration

Build once and promote the same backend image digest and frontend artifact checksum. If image copies are required across AWS accounts, verify the destination digest and registry permissions. Store release artifacts in a durable controlled registry or artifact store, with retention sufficient for deployment and rollback; short-lived CI artifacts alone may expire.

Environment configuration changes at deployment/runtime, not by rebuilding the artifact. Backend secrets are fetched or provisioned through the approved secret mechanism on the target host. A public frontend runtime configuration may contain the API base URL and non-secret flags only. The SPA must implement and test loading this configuration; it is not present in the current reference workflows. Never place API keys or credentials in frontend files.

8 Security and tech lead controls

- Use GitHub OIDC to assume short-lived AWS roles; create separate least-privilege roles for each environment and application. Scope trust to the correct repository and GitHub Environment; staging and production deploy repositories need their own trust subjects.
- Protect deployment repositories, workflow files, CODEOWNERS, environment rules, SSM deployment documents and artifact registries from unauthorized modification. Restrict who can merge or bypass rules.
- Do not expose AWS credentials to untrusted pull-request code. Apply dependency review, secret scanning, image scanning and audit logging as supported by the chosen tools.
- The tech lead validates the release manifest and evidence before merging into both staging and production. A role-based delegate may substitute only under the approved team policy.

GitHub Environment reviewer availability depends on the organization plan and repository visibility. Verify the actual entitlement. If unavailable, use an approved external deployment gate; do not claim an unsupported control is active.

9 Example release

Stage	Example action and evidence

Stage	Example action and evidence
DEV	Commit abc123… is built; image sha256:… and frontend checksum are recorded. Dev smoke tests pass.
STAGING	Tech lead approves PR STG-42 with those artifact IDs; staging deployment and E2E tests pass.
PRODUCTION	Tech lead approves PR PROD-17 with the same IDs and staging evidence; deployment and public smoke test pass.
MONITOR	Owner checks error rate, latency and business transaction health; records release status or rolls back.

All IDs are placeholders for explanation. Record real immutable digests, checksums, PR URLs, test runs, approver names and deployment timestamps for each release.

10 Setup checklist for DevOps and tech leads

- Create the three repositories, AWS environments, scoped roles, S3 buckets, CDN distributions, EC2/SSM hosts and environment-specific secrets.
- Configure protected branches, CODEOWNERS and required tech-lead approval on the base branches of staging and production repositories.
- Implement and test the cross-repository promotion workflows and staging evidence gate; verify that a failed DEV or STAGING test blocks the next stage.
- Add immutable artifact storage, release manifests, digest verification, least-privilege cross-account access and artifact retention.
- Implement production environment gates where supported; confirm branch policy, protection rules and emergency access procedures.
- Test successful promotion, wrong artifact, failed staging, missing approval, rollback and customer-facing smoke tests in non-production.

Current limitation: none of the three-environment promotion repositories, gates or AWS resources have been created or tested as part of this guide. The provided application code and AWS account were not available. This is an implementation manual and design, not a production-readiness certification.

11 References

GitHub protected branches: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

GitHub CODEOWNERS: https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners

GitHub OIDC for AWS: https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-aws

AWS DevOps environments: https://docs.aws.amazon.com/prescriptive-guidance/latest/choosing-git-branch-approach/understanding-the-dev-ops-environments.html